Intro to Java, Classes and Objects

Prof. Guilherme Cavalcanti

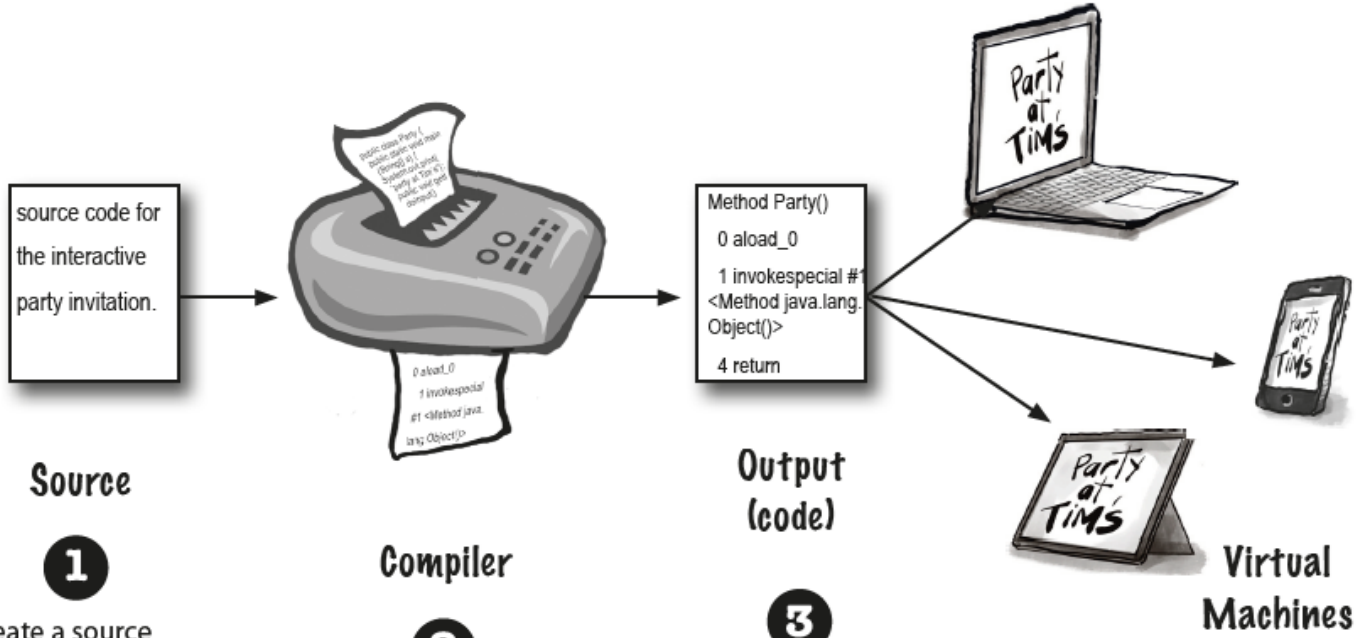
A very brief history of Java

- Java was initially released, on January 23, 1996
- Since than, tons and tons of millions of Java code
- You will most certainly come across Java code that’s quite old
- Java is famous for its backward compatibility

A very brief history of Java

- Java is fast as languages like C and Rust
- Java has a magic super-power—the JVM (Java Virtual Machine)
- But, compared to C and Rust, Java uses a lot of memory.

The way Java works



Source

1

Create a source document. Use an established protocol (in this case, the Java language).

Compiler

2

Run your document through a source code compiler. The compiler checks for errors and won't let you compile until it's satisfied that everything will run correctly.

Output (code)

3

The compiler creates a new document, coded into Java ***bytecode***. Any device capable of running Java will be able to interpret/translate this file into something it can run. The compiled bytecode is platform-independent.

Virtual Machines

4

Your friends all have a Java ***virtual*** machine (JVM), implemented in software, running inside their electronic gadgets. When your friends run your program, the virtual machine reads and runs the bytecode.

What you'll do in Java

```
import java.awt.*;
import java.awt.event.*;

class Party {
 public void buildInvite() {
  Frame f = new Frame();
  Label l = new Label("Party at Tim's");
  Button b = new Button("You bet");
  Button c = new Button("Shoot me");
  Panel p = new Panel();
  p.add(l);
 }  // more code here...
}
```

Source

1

Type your source code.

Save as: ***Party.java***

```
File  Edit  Window  Help  Plead
%javac Party.java
```

Compiler

2

Compile the ***Party.java*** file by running `javac` (the compiler application). If you don't have errors, you'll get a second document named ***Party.class***.

The compiler-generated Party.class file is made up of *bytecodes*.

```
Method Party()
  0 aload_0
  1 invokespecial #1 <Method
java.lang.Object()>
  4 return
Method void buildInvite()
  0 new #2 <Class java.awt.Frame>
  3 dup
  4 invokespecial #3 <Method
java.awt.Frame()>
```

Output (code)

3

Compiled code: ***Party.class***

```
File  Edit  Window  Help  Swear
%java Party
```

Virtual Machines

4

Run the program by starting the Java Virtual Machine (JVM) with the ***Party.class*** file. The JVM translates the *bytecode* into something the underlying platform understands, and runs your program.

Exercise: try to guess what each line of code is doing.

```
int size = 27;
String name = "Fido";
Dog myDog = new Dog(name, size);
x = size - 5;
if (x < 15) myDog.bark(8);

while (x > 3) {
  myDog.play();
}
```

declare an integer variable named 'size' and give it the value 27
if x (value of 22) is less than 15, tell the dog to bark 8 times

Exercise: try to guess what each line of code is doing

```
int[] numList = {2, 4, 6, 8};
System.out.print("Hello");
System.out.print("Dog: " + name);
String num = "8";
int z = Integer.parseInt(num);

try {
  readTheFile("myFile.txt");
}
catch (FileNotFoundException ex) {
  System.out.print("File not found.");
}
```

print out "Hello"... probably at the command line

Solution

```
int size = 27;
String name = "Fido";
Dog myDog = new Dog(name, size);
x = size - 5;
if (x < 15) myDog.bark(8);

while (x > 3) {
  myDog.play();
}
```

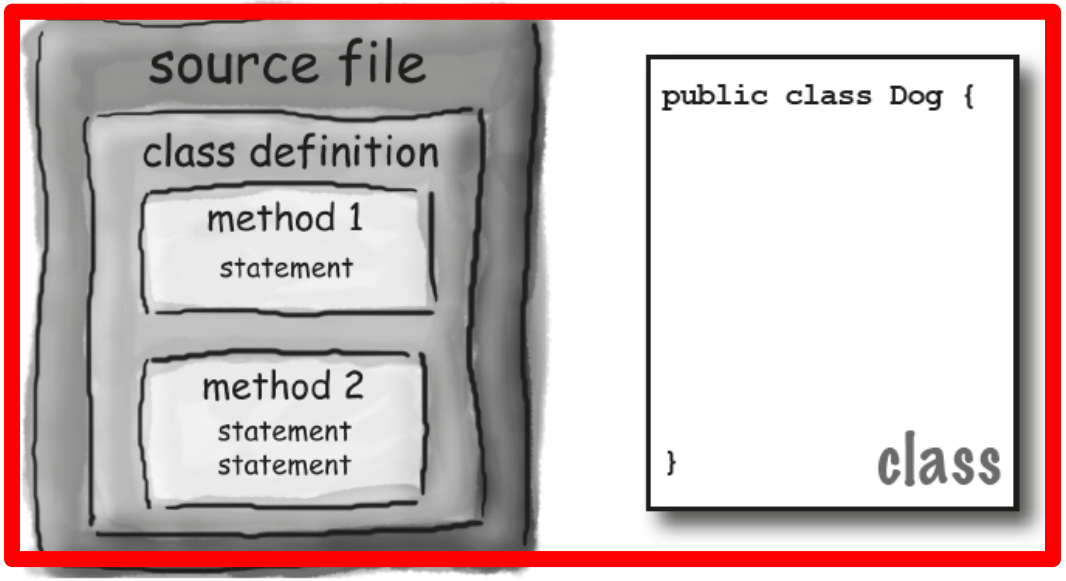
declare an integer variable named 'size' and give it the value 27
declare a string of characters variable named 'name' and give it the value "Fido"
declare a new Dog variable 'myDog' and make the new Dog using 'name' and 'size'
subtract 5 from 27 (value of 'size') and assign it to a variable named 'x'
if x (value of 22) is less than 15, tell the dog to bark 8 times
keep looping as long as x is greater than 3...
tell the dog to play (whatever THAT means to a dog...)
this looks like the end of the loop -- everything in { } is done in the loop

Solution continuation

```
int[] numList = {2, 4, 6, 8};
System.out.print("Hello");
System.out.print("Dog: " + name);
String num = "8";
int z = Integer.parseInt(num);

try {
  readTheFile("myFile.txt");
}
catch (FileNotFoundException ex) {
  System.out.print("File not found.");
}
```

declare a list of integers variable 'numList', and put 2,4,6,8 into the list.
print out "Hello"... probably at the command line
print out "Dog: Fido" (the value of 'name' is "Fido") at the command line
declare a character string variable 'num' and give it the value of "8"
convert the string of characters "8" into an actual numeric value 8
try to do something...maybe the thing we're trying isn't guaranteed to work...
read a text file named "myFile.txt" (or at least TRY to read the file...)
must be the end of the "things to try", so I guess you could try many things...
this must be where you find out if the thing you tried didn't work...
if the thing we tried failed, print "File not found" out at the command line
looks like everything in the { } is what to do if the 'try' didn't work...

Code structure in Java




```
public class Dog {

}
```

class

```
public class Dog {
   void bark() {

   }
}
```

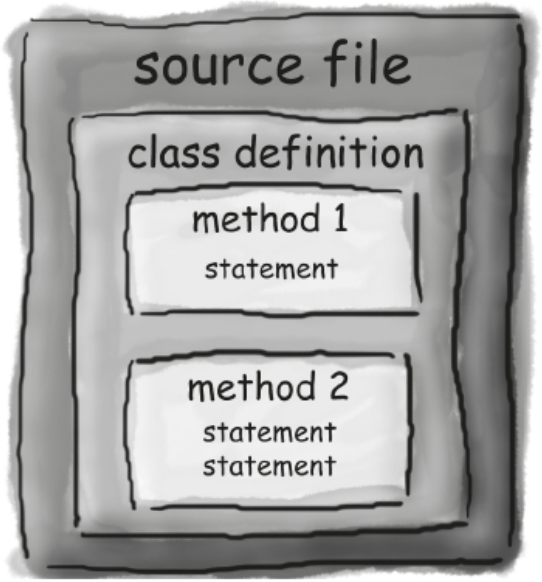
method

```
public class Dog {
   void bark() {
    statement1;
    statement2;
   }
}
```

statements

In a source file, put a class.

Code structure in Java




```
public class Dog {

}
```

class

```
public class Dog {
   void bark() {

   }
}
```

method

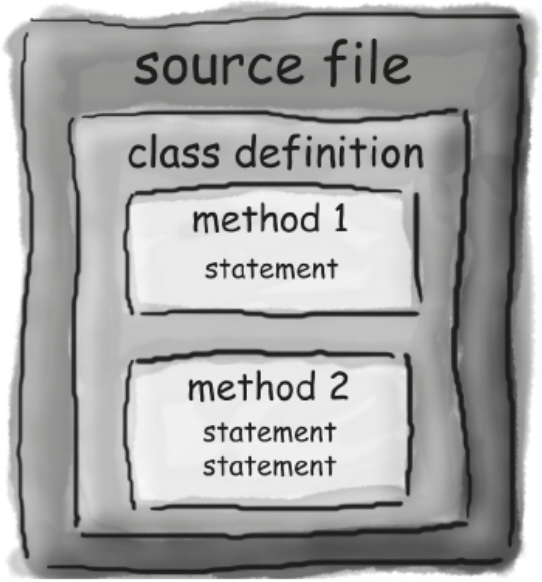
```
public class Dog {
   void bark() {
    statement1;
    statement2;
   }
}
```

statements

In a source file, put a class.

In a class, put methods.

Code structure in Java




```
public class Dog {

}
```

class

```
public class Dog {
   void bark() {

   }
}
```

method

```
public class Dog {
   void bark() {
    statement1;
    statement2;
   }
}
```

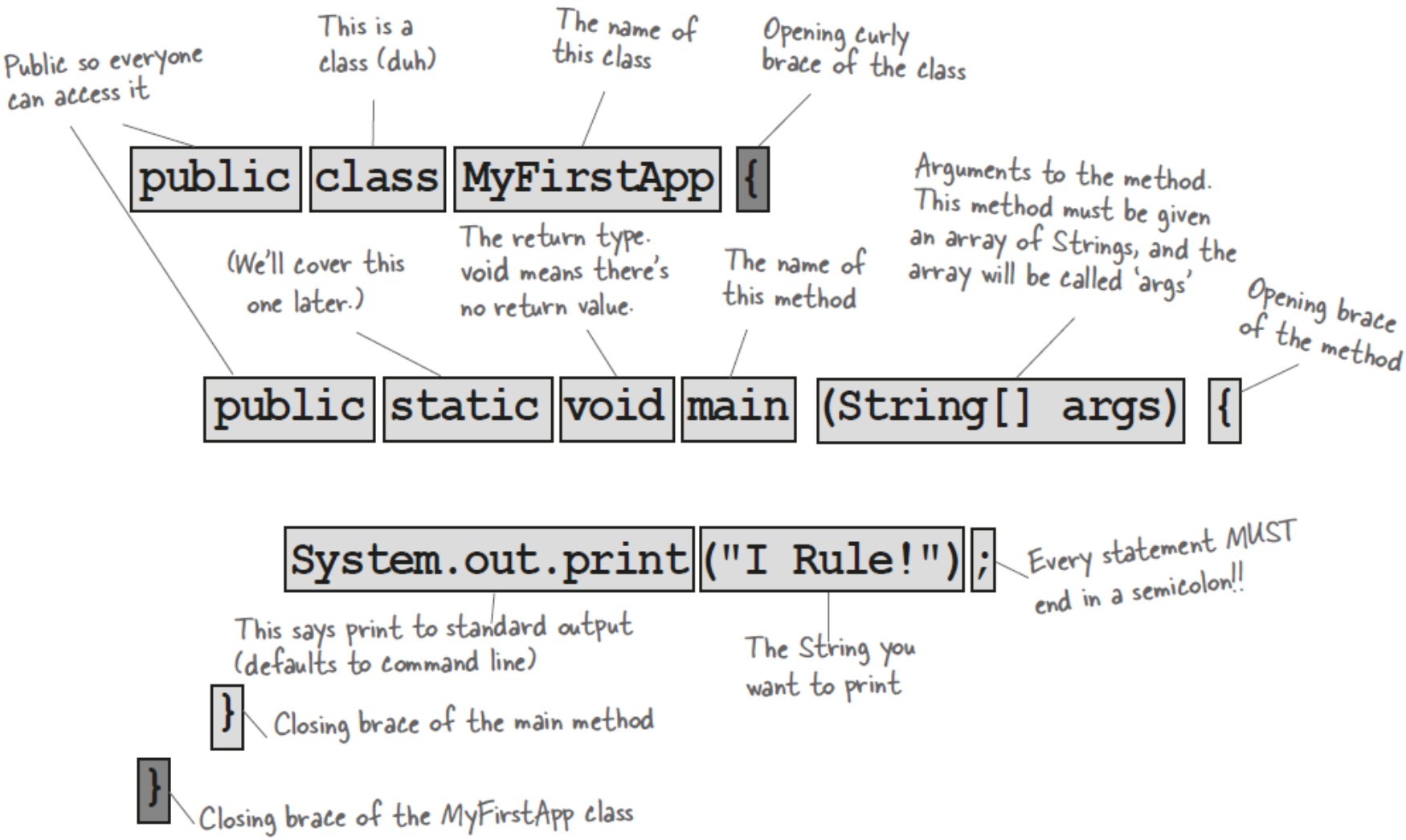
statements

In a source file, put a class.

In a class, put methods.

In a method, put statements.

Anatomy of a Java class



Writing a class with a ***main()***

```
public class MyFirstApp {
  public static void main (String[] args) {
    System.out.print("I Rule!");
    System.out.println("The World");
  }
}
```

MyFirstApp.java

```
Compiled from "MyFirstApp.java"
public class ch1.MyFirstApp {
  public ch1.MyFirstApp();
    Code:
       0: aload_0
       1: invokespecial #1
// Method java/lang/Object."<init>":()V
       4: return
  public static void main(java.lang.
String[]);
```

MyFirstApp.class

```
File  Edit  Window  Help  Scream
%java MyFirstApp
I Rule!
The World
```

❶ Save

`MyFirstApp.java`

❷ Compile

`javac MyFirstApp.java`

❸ Run

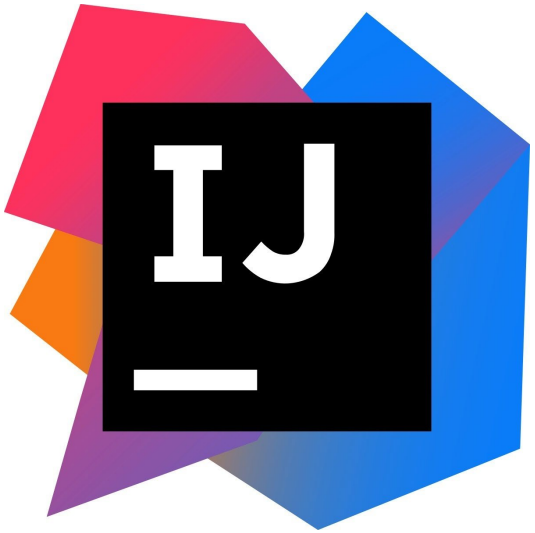
`java MyFirstApp`

```
public class MyFirstApp {

   public static void main (String[] args) {
     System.out.println("I Rule!");
     System.out.println("The World");
   }

}
```

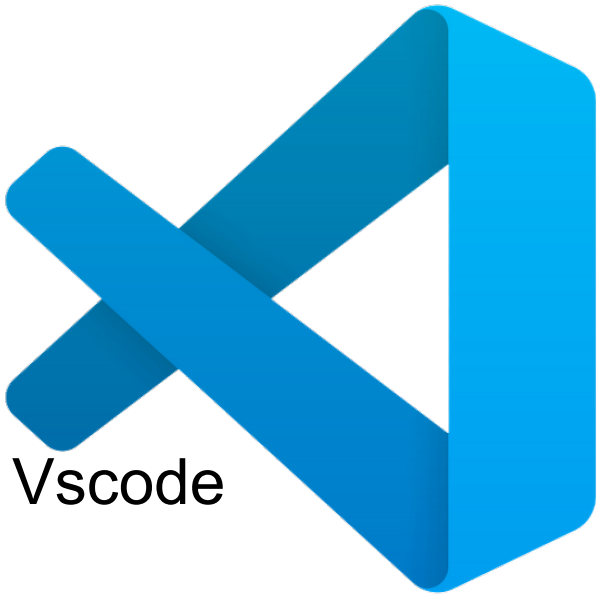
No matter how many classes your program uses, there's got to be a `main()` method to get the ball rolling.

In Java, everything is about ***classes***!

IDEs (Integrated Development Environments)

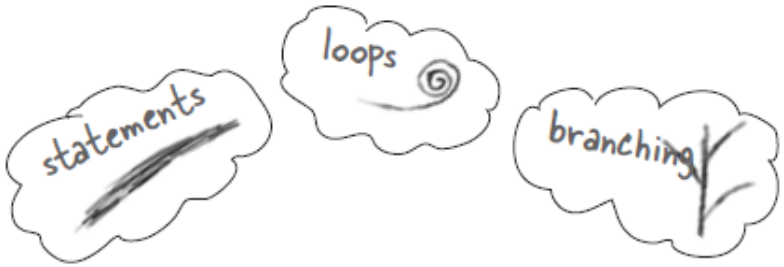


Intellij



Vscode

What can you do in the ***main (or any)*** method?



1 do something

Statements: declarations, assignments, method calls, etc.

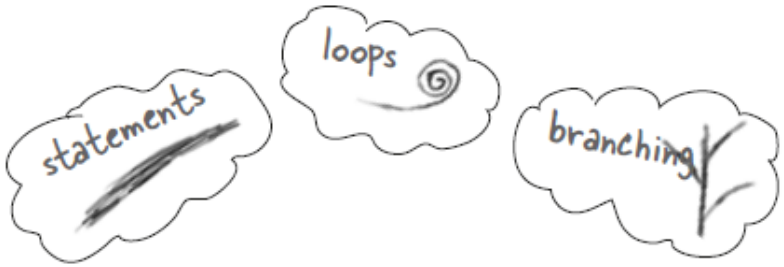
```
int x = 3;
String name = "Dirk";
x = x * 17;
System.out.print("x is " + x);
double d = Math.random();
// this is a comment
```

2 do something again and again

Loops: *for* and *while*

```
while (x > 12) {
  x = x - 1;
}

for (int i = 0; i < 10; i = i + 1) {
  System.out.print("i is now " + i);
}
```

What can you say in the main method?



3 do something under this condition

Branching: *if/else* tests

```
if (x == 10) {
  System.out.print("x must be 10");
} else {
  System.out.print("x isn't 10");
}
if ((x < 3) && (name.equals("Dirk"))) {
  System.out.println("Gently");
}
System.out.print("this line runs no matter what");
```

Important Syntax Tips (never forget!)

- Each statement must end in a semicolon.

```
x = x + 1;
```

- A single-line comment begins with two forward slashes.

```
x = 22;
// this line disturbs me
```

- Most white space doesn't matter.

```
x      =      3 ;
```

Important Syntax Tips (never forget!)

- Variables are declared with a name and a type (you'll learn about the Java types soon).

```
int weight;

//type: int, name: weight
```

- Classes and methods must be defined within a pair of curly braces.

```
public void go() {
   // amazing code here
   }
```

Looping: *while*, *do-while*, *for*

```
while (moreBalls == true) {
  keepJuggling();
}
```

- < (less than)
- > (greater than)
- == (equality) (yes, that's two equals signs)
- == is not =

Simple boolean tests

```java
public class Loopy {
  public static void main(String[] args) {
    int x = 1;
    System.out.println("Before the Loop");
    while (x < 4) {
      System.out.println("In the loop");
      System.out.println("Value of x is " + x);
      x = x + 1;
    }
    System.out.println("This is after the loop");
  }
}
```

```
% java Loopy
Before the Loop
In the loop
Value of x is 1
In the loop
Value of x is 2
In the loop
Value of x is 3
This is after the loop
```

This is the output

Example *while loop*

Conditional branching

```
class IfTest {
  public static void main (String[] args) {
    int x = 3;
    if (x == 3) {
      System.out.println("x must be 3");
    }
    System.out.println("This runs no matter what");
  }
}
```

```
% java IfTest
x must be 3
This runs no matter what
```

Code output

```
class IfTest2 {
  public static void main(String[] args) {
    int x = 2;
    if (x == 3) {
      System.out.println("x must be 3");
    } else {
      System.out.println("x is NOT 3");
    }
    System.out.println("This runs no matter what");
  }
}
```

New output

```
% java IfTest2
x is NOT 3
This runs no matter what
```

Classes and Objects

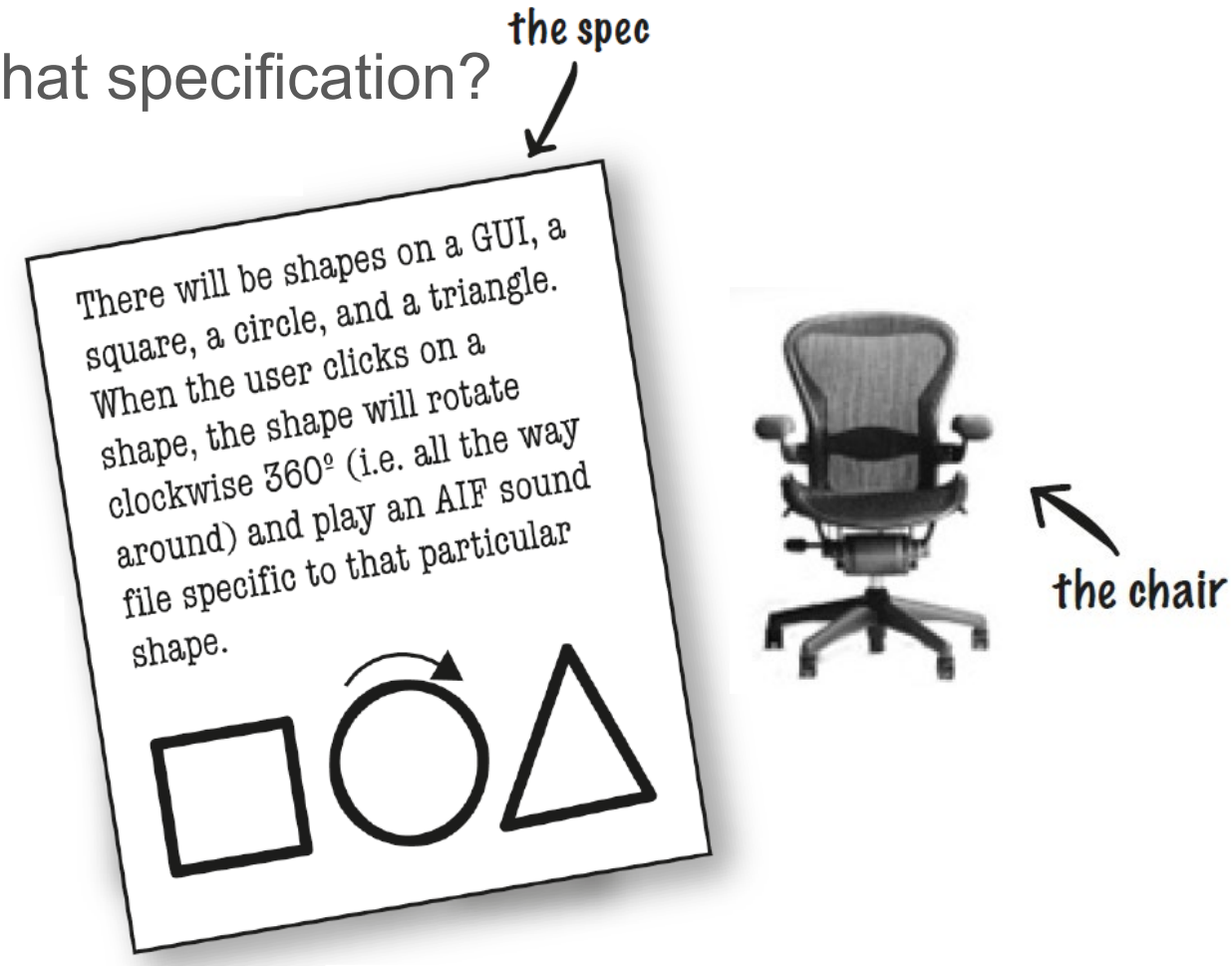
Let’s leave the **procedural** world

- What makes object-oriented programming different
- Difference between classes and objects
- How that can make a better programmer

Warning! There is no comeback.

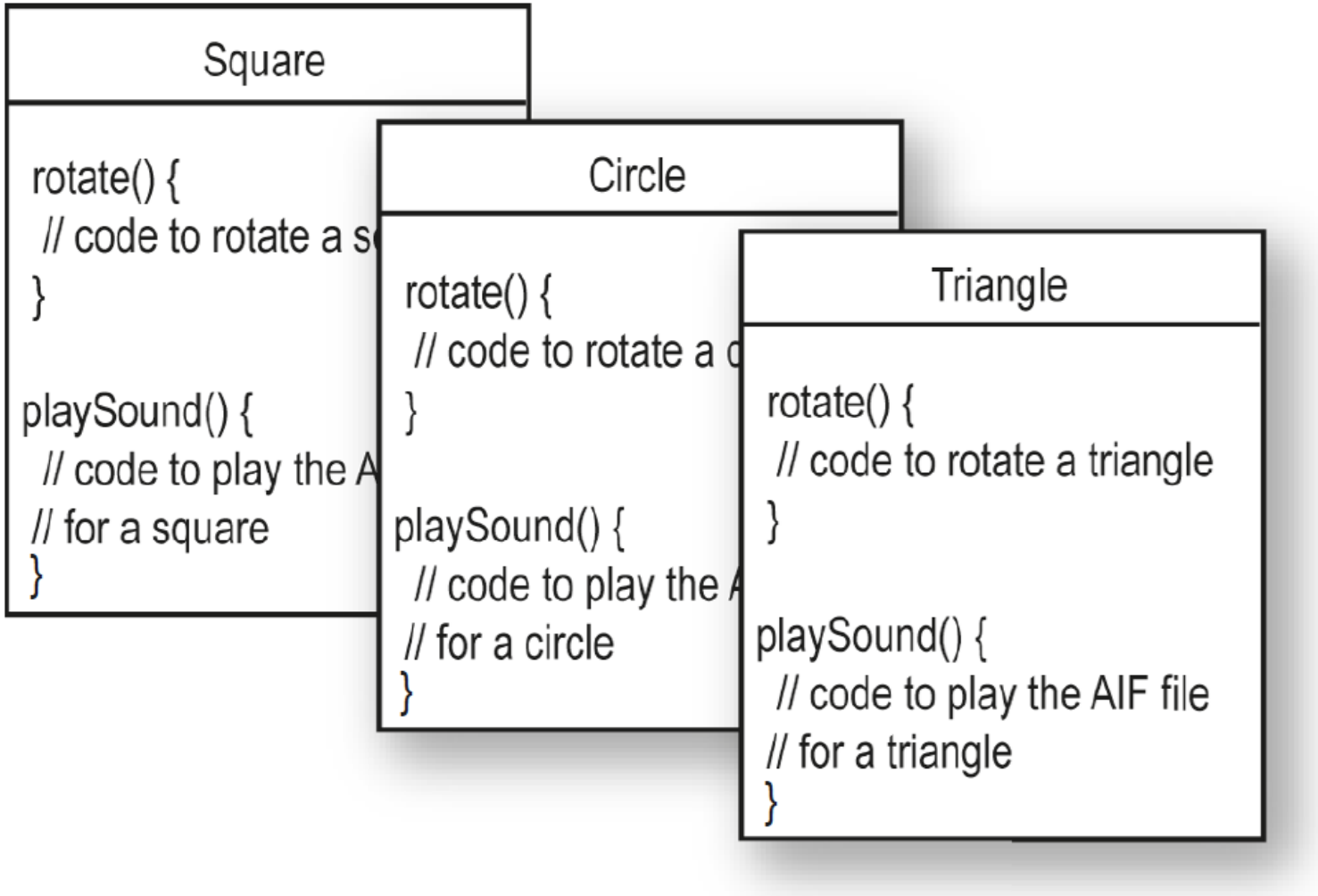
How Objects Can Change Your Life

- How to code that specification?



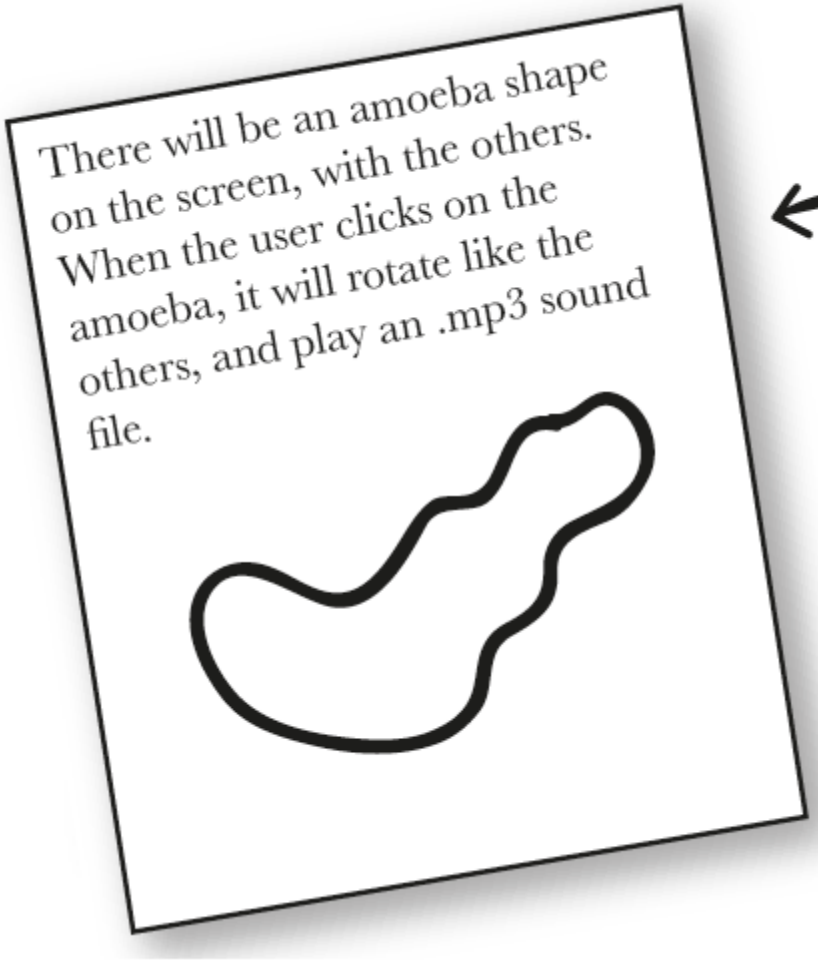
```
rotate(shapeNum) {
   // make the shape rotate 360°
}
playSound(shapeNum) {
   // use shapeNum to lookup which shape
   // AIF sound to play, and play it
}
```

Laura’s idea: *write important procedures*



Brad's idea: *write a class for each of shape*

Which solution is better for you?



what got added to the spec

But wait! There’s been a spec change.

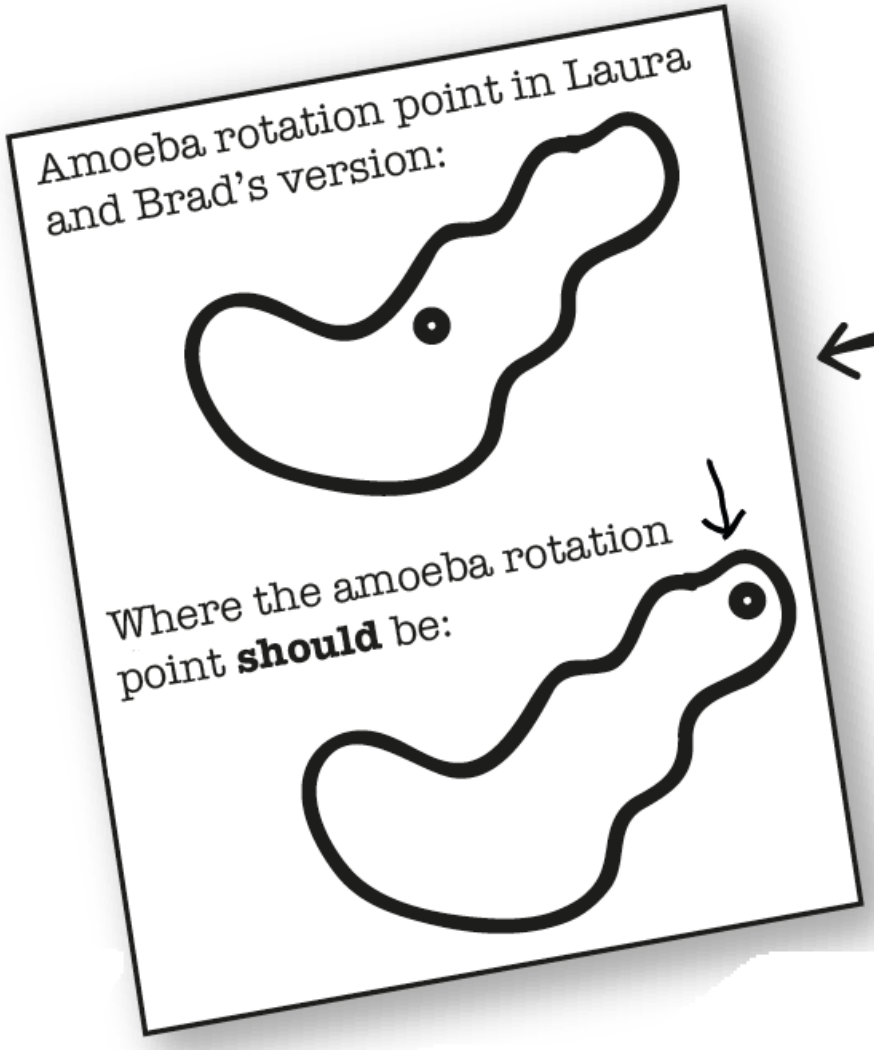
```
playSound(shapeNum) {
   // if the shape is not an amoeba,
   // use shapeNum to lookup which
   // AIF sound to play, and play it
   // else
   // play amoeba .mp3 sound
}
```

Laura’s idea: *change* ***playSound***

Amoeba

```
rotate() {
 // code to rotate an amoeba
 }

playSound() {
 // code to play the new
 // .mp3 file for an amoeba
 }
```

Brad's idea: *write a class for* ***Amoeba***



What the spec conveniently forgot to mention

Oh, no, that's not how the amoeba is supposed to rotate...

```
rotate(shapeNum, xPt, yPt) {
    // if the shape is not an amoeba,
        // calculate the center point
        // based on a rectangle,
        // then rotate
    // else
        // use the xPt and yPt as
        // the rotation point offset
        // and then rotate
}
```

Laura’s idea: *change* ***rotate***

Amoeba

```
int xPoint
int yPoint

rotate() {
  // code to rotate an amoeba
  // using amoeba's x and y
  }

playSound() {
  // code to play the new
  // .mp3 file for an amoeba
  }
```

Brad's idea: *modify Amoeba class*

***LAURA**: You've got duplicated code! The rotate procedure is in all four Shape things.*

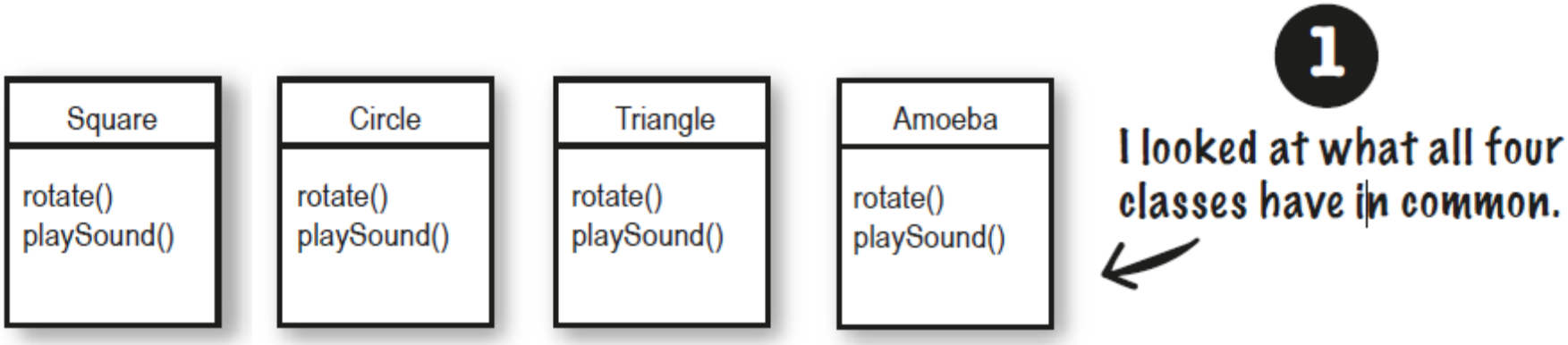
***BRAD**: It's a method, not a procedure. And they're classes, not things.*

***LAURA**: Whatever. It's a stupid design. You have to maintain four different rotate "methods." How can that ever be good?*

***BRAD**: Oh, I guess you didn't see the final design. Let me show you how <u>OO inheritance</u> works, Laura.*

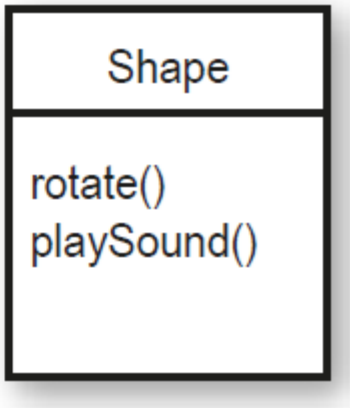
The OO solution is better, right?

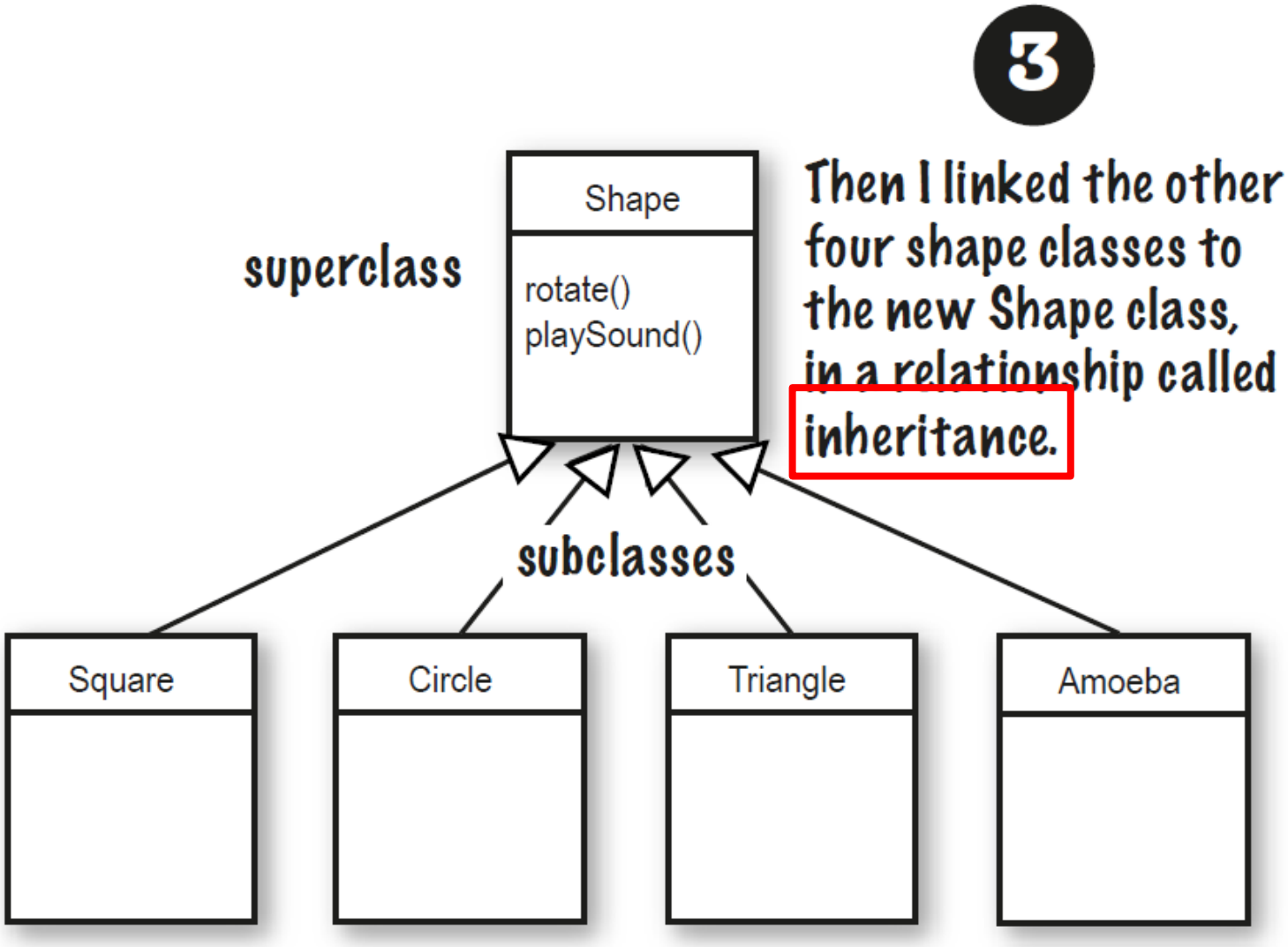
The OO Solution

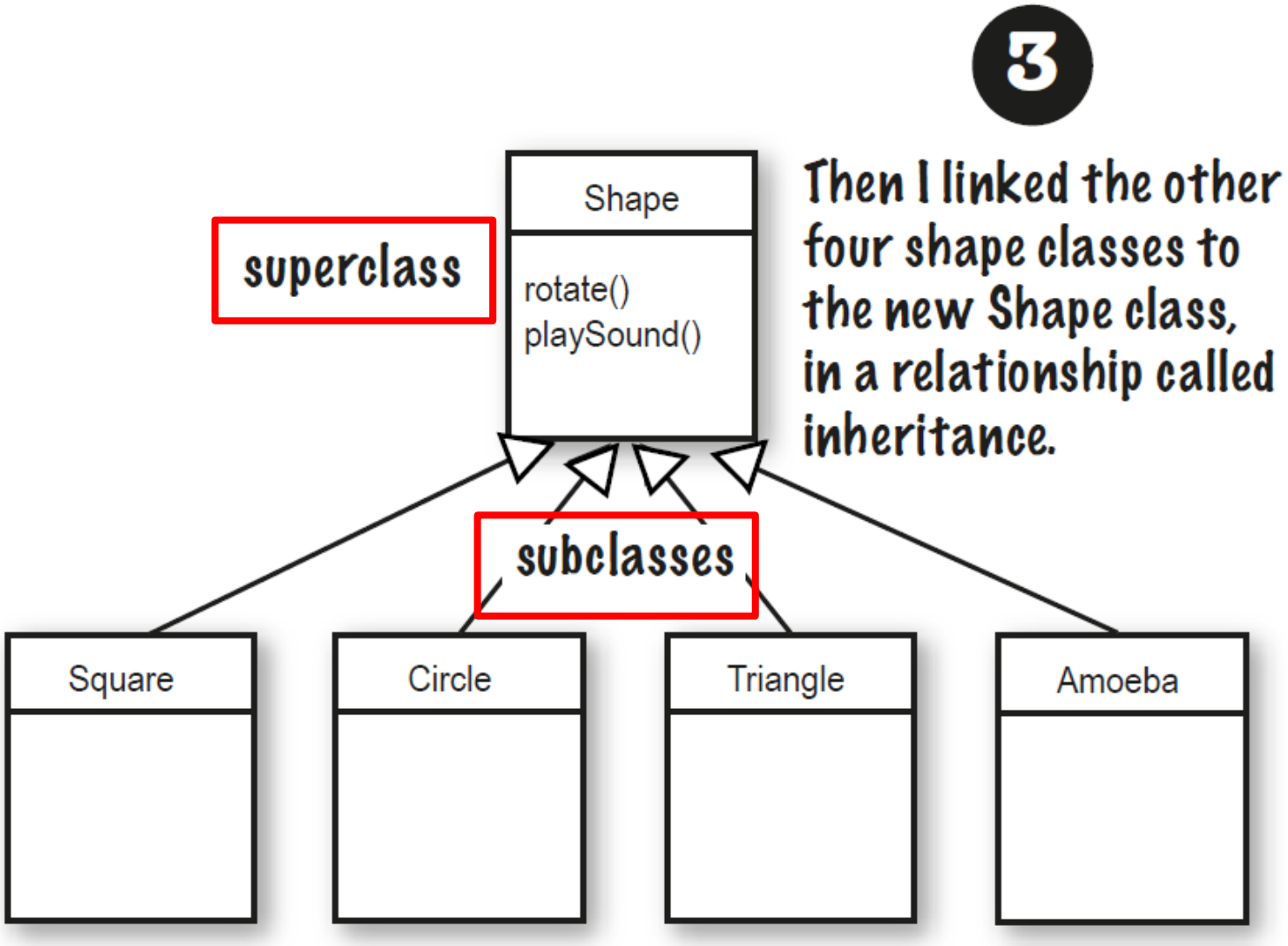


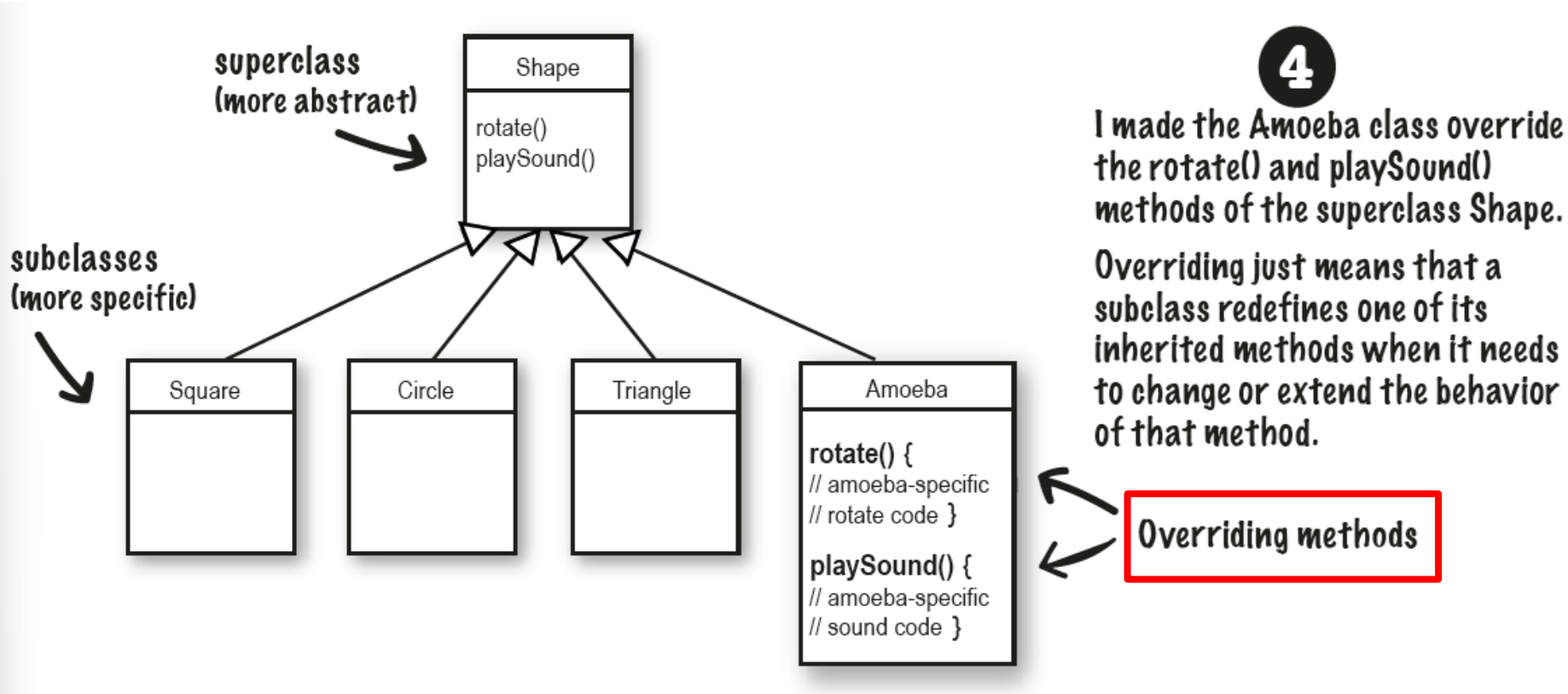
2

They're Shapes, and they all rotate and playSound. So I abstracted out the common features and put them into a new class called Shape.




3
Then I linked the other four shape classes to the new Shape class, in a relationship called inheritance.
superclass
Shape
rotate()
playSound()
subclasses
Square
Circle
Triangle
Amoeba


3
Then I linked the other four shape classes to the new Shape class, in a relationship called inheritance.
superclass
Shape
rotate()
playSound()
subclasses
Square
Circle
Triangle
Amoeba



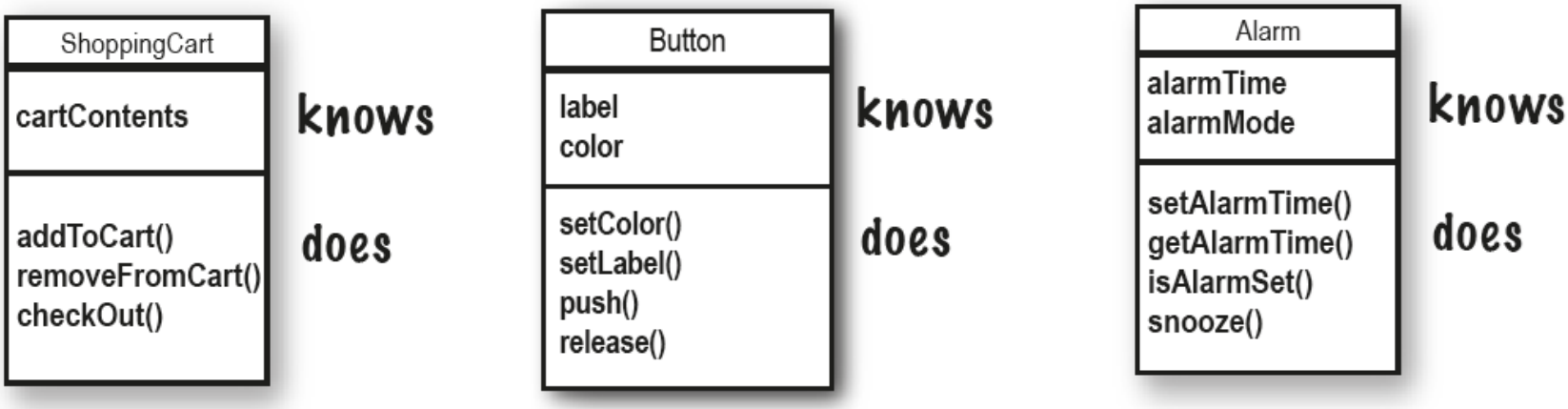
What about the Amoeba rotate()?

What will you might like about OO?

- It will help you design in a more natural way. Things have a way of evolving.
- Not messing around with code you've already tested, just to add a new feature.
- You might like that the data and the methods that operate on that data are together in one class.
- Reusing code in other applications. When you write a new class, you can make it flexible enough to be used in something new, later.

Thinking about ***classes*** and ***objects***. Think about:

- things the object **knows**
- things the object **does**



Things an object *knows* about itself are called

- instance variables **or attributes**

instance variables
(state)

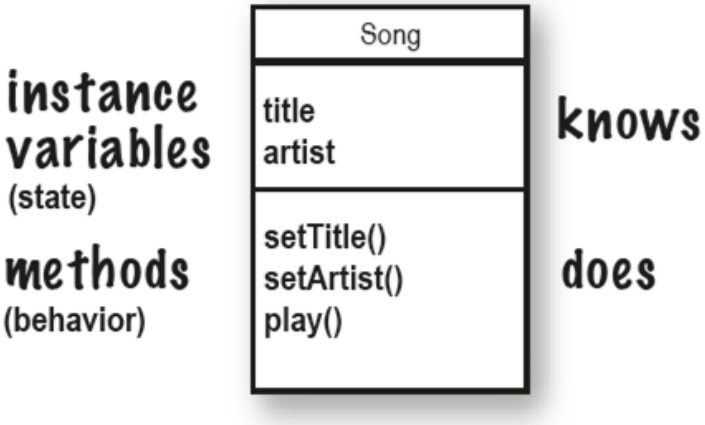
knows

Things an object *knows* about itself are called

- instance variables **or attributes**

Things an object can *do* are called

- methods

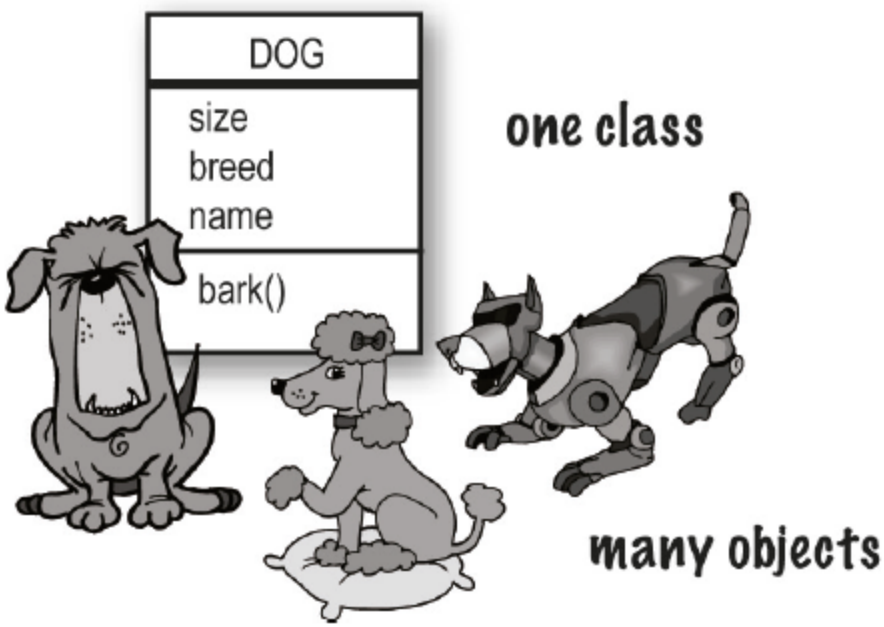


What's the difference between a class and an object?

A class is not an object

(but it's used to construct them)

What's the difference between a class and an object?



A class is not an object

(but it's used to construct them)

- A **class** is a blueprint for an object. It tells the virtual machine how to make an object of that particular type.

- Each **object** made from that class can have its own values for the instance variables/attributes of that class.

Example: contacts list (class), entry on that list (object)

In a true OO application, you need objects **talking** to other objects.

Java Garbage Collector

- Every object created goes into an area of memory known as The Heap, called the Garbage Collectible Heap.

Java Garbage Collector

- Java allocates memory space on the heap according to how much that particular object needs.

Java Garbage Collector

- Java manages that memory for you, to reclaim that space.

Exercise: making your first object

1 Write your class

```
class Dog {
  int size;
  String breed;
  String name;

  void bark() {
    System.out.println("Ruff! Ruff!");
  }
}
```

Instance variables

A method

Exercise: making your first object

1 Write your class

```
class Dog {
  int size;
  String breed;
  String name;

  void bark() {
    System.out.println("Ruff! Ruff!");
  }
}
```

Instance variables

A method

2 Write a tester (TestDrive) class

```
class DogTestDrive {
  public static void main(String[] args) {
    // Dog test code goes  here
  }
}
```

Just a main method

(we're gonna put code in it in the next step)

3 In your tester, make an object and access the object's variables and methods

```
class DogTestDrive {
  public static void main(String[] args) {
    Dog d = new Dog();
    d.size = 40;
    d.bark();
  }
}
```

Make a Dog object

Use the dot operator (.) to set the size of the Dog

and to call its bark() method

Dot operator

The Dot Operator (.)

The dot operator (.) gives you access to an object's state and behavior (instance variables and methods).

```
// make a new object
Dog d = new Dog();

// tell it to bark by using the
// dot operator on the
// variable d to call bark()
d.bark();

// set its size using the
// dot operator
d.size = 40;
```

Exercise: making and testing movie objects

Given:

```
public class MovieTestDrive {
  public static void main(String[] args) {
    Movie one = new Movie();
    one.title = "Gone with the Stock";
    one.genre = "Tragic";
    one.rating = -2;
    Movie two = new Movie();
    two.title = "Lost in Cubicle Space";
    two.genre = "Comedy";
    two.rating = 5;
    two.playIt();
    Movie three = new Movie();
    three.title = "Byte Club";
    three.genre = "Tragic but ultimately uplifting";
    three.rating = 127;
  }
}
```